TURN_PROMPT_DESIGN.md

Turn Prompt Generation — Design Specification

Overview

Each character receives one self-contained prompt per turn, generated fresh by the context builder in initiative order. Prompts are never patched or appended — they are assembled from scratch at the moment of each character's action, incorporating all scene state changes from characters who acted earlier in the same turn.

This mirrors tabletop play: Player A attacks Orc A, then Player B attacks Orc B — unless Player A missed, in which case Orc A is still standing when Player B acts. Sequential actions within a turn create cascading state. Each actor sees the consequences of everyone who went before them.

Core Principle: Lazy Sequential Assembly

Prompts are generated in initiative order. Each prompt incorporates the scene state as it exists at the moment of that character's action — not at the start of the turn.

```
Turn N:
  Roll initiative for all active characters in scene
  Order characters by initiative result

  For each character in order:
    1. Assemble scene state (current — post previous actions)
    2. Filter to character's observable information
    3. Generate available actions from context
    4. Build prompt
    5. Send to LLM (or human via OpenClaw)
    6. Receive response
    7. Update scene state
    8. Proceed to next character

  Turn N complete:
    DM-Left updates all condition vectors
```

```
DM-Right generates narration
Turn N+1 begins
```

Prompt Structure

Every character prompt contains these sections in order:

```
SYSTEM          — role instruction and output format
CHARACTER       — name, race, class, brief description
SOUL SUMMARY    — condensed from SOUL.md (stable, never changes)
CURRENT STATE   — rendered neurochemical condition vector
YOUR LOCATION   — current zone within the scene
SCENE STATE     — filtered observable information at this moment
WHAT CHANGED    — what happened before your turn this round
AVAILABLE ACTIONS — contextually generated from zone + situation
OUTPUT FORMAT   — INTERNAL / BODY LANGUAGE / SPOKEN / ACTION
```

Information Filtering

The DM holds an omniscient scene state. Each character receives a filtered view based on:

What determines visibility:

- **Zone —** characters can observe adjacent zones partially, their own zone fully, non-adjacent zones not at all
- **Perception —** high acetylcholine expands observable detail
- **Special abilities —** Speak with Animals, Detect Magic etc. reveal additional condition vector dimensions
- **Prior knowledge —** what the character established before this scene (Fennel's 3-day glade experience)

What is never shared:

- Other characters' INTERNAL monologue
- DM-only notes from PLACE.md and magical_items.md
- Condition vector raw numbers of other characters

- Information from scenes the character was not present for

Zone visibility matrix:

```
Your zone     Can observe fully    Can observe partially
TRAIL         TRAIL                TREELINE (20 ft)
TREELINE      TREELINE             TRAIL (20 ft), GLADE (full)
GLADE         GLADE                TREELINE (full), POOL (full)
POOL          POOL                 GLADE (full)
```

Available Actions Generation

Actions are generated contextually — not from a generic list. Every action must be physically possible from the character's current zone and situation.

Action categories (all optional except INTERNAL and BODY LANGUAGE):

```
MOVE        — zone transitions (one zone per ACTION)
OBSERVE     — perception, investigation, tracking checks
INTERACT    — items, environment, other characters
PREPARE     — ready weapons, spells, abilities
ABILITY     — class abilities, spells, special skills
SOCIAL      — speak, signal, gesture (if someone present)
WAIT        — hold, go still, observe passively
```

Contextual generation rules:

- MOVE options list only adjacent zones
- OBSERVE options reference only what is detectable from current zone
- INTERACT options reference only items present in zone (from CONTENTS.md)
- ABILITY options reference only what the character currently has available (spell slots, uses per rest etc.)
- SOCIAL options reference only characters present in observable range
- Actions that are impossible are never listed (e.g. Speak with Animals when no animals present, attack when no target in range)

Special case — creatures without language:

- SPOKEN is omitted from format entirely
- ACTION vocabulary is constrained to predatory/survival options appropriate to the creature type
- INTERNAL uses sensation-language instructions

The WHAT CHANGED Section

This is the key mechanism for sequential state propagation within a turn. It tells each character explicitly what happened before their action this round.

Format:

```
WHAT CHANGED THIS ROUND:
  [Summary of actions taken by characters with higher
   initiative, and their observable consequences]
```

Filtering rules:

- Only include changes observable from the character's zone
- Never reveal INTERNAL of other characters
- Describe consequences, not intentions
- If nothing observable changed: "Nothing. [Situation summary]."

Examples:

Character went before you, their action was observable:

```
WHAT CHANGED THIS ROUND:
  Aeliana remained on the TRAIL and studied the shimmer.
  She has not moved. TRAIL remains empty from your position.
```

Character went before you, their action created new state:

```
WHAT CHANGED THIS ROUND:
  A second heat-shape arrived at the TREELINE from the TRAIL.
  It moved carefully into the boundary zone — just inside
  the safe side. It is still now. It knows something is here.
```

Nothing changed:

```
WHAT CHANGED THIS ROUND:
  Nothing. Fennel in GLADE. TRAIL empty. Boundary holds.
```

Condition Vector in Prompts

The condition vector is rendered as annotated natural language. Raw numbers plus a brief description of what each value means relative to the character's baseline.

```
CURRENT STATE:
  dopamine:       0.88  (above baseline — reward circuits active)
  cortisol:       0.28  (low — no immediate threat detected)
  norepinephrine: 0.60  (focus sharpening)
  endorphins:     0.75  (physically good)
  serotonin:      0.70  (stable)
  acetylcholine:  0.80  (tracking mode — noticing everything)
  oxytocin:       0.55  (baseline)
```

Annotations are generated by comparing current value to baseline and selecting from a description table per dimension:

```
ANNOTATIONS = {
    "dopamine": {
        (0.0, 0.3): "well below baseline — disengaged, apathetic",
        (0.3, 0.5): "below baseline — low motivation",
        (0.5, 0.7): "near baseline",
        (0.7, 0.85): "above baseline — reward circuits active",
        (0.85, 1.0): "well above baseline — strongly approach-motivated",
    },
    "cortisol": {
        (0.0, 0.2): "very low — fully at ease",
        (0.2, 0.4): "low — no immediate threat detected",
        (0.4, 0.6): "moderate — alert, some threat awareness",
        (0.6, 0.8): "elevated — threat response active",
        (0.8, 1.0): "high — acute stress, tunnel vision",
    },
    # ... etc for all seven dimensions
}
```

Scene Activation and Lazy Evaluation

Scenes only run turns when a player character is present. Characters in inactive scenes are frozen at last known state. Characters that have never been observed are indeterminate — their condition vector does not exist until first activation.

```
class CharacterState(Enum):
    ACTIVE        = "active"       # in active scene, taking turns
    FROZEN        = "frozen"       # inactive scene, state preserved
    INDETERMINATE = "indeterminate" # never observed, state unresolved
```

Indeterminate characters:

When a player enters a scene containing an indeterminate character, the context builder samples their initial condition vector from a distribution:

```
def animate_character(soul, context) -> NeurochemicalState:
    # Sample each dimension from distribution
    # centered on baseline, shaped by context modifiers
    return NeurochemicalState(
        dopamine = sample(
            mean=soul.baseline.dopamine,
            std=soul.volatility.dopamine * 0.3,
            modifiers=context.dopamine_modifiers
        ),
        # ... etc
    )
```

Context modifiers are derived from the scene and place files:

```
# Example: Echoing Caverns, unknown creature, intruders present
context.modifiers = {
    "intruders_detected": {"cortisol": +0.30},
    "ancient_territory":  {"serotonin": +0.20},
    "darkness_complete":  {"dopamine": +0.15},
    "torch_light":        {"cortisol": +0.10},
}
```

This produces natural variation across playthroughs while remaining contextually coherent. The same creature animates slightly differently each time — same SOUL.md, different sample, different story.

Human Players

Human players via OpenClaw/WhatsApp receive the same filtered scene context as LLM agents, minus the CURRENT STATE section (their neurochemical state is not shown to them — it informs DM-Right's response to them, not their own output).

Their response is received as free text and parsed into:

- BODY LANGUAGE (inferred from description)
- SPOKEN (extracted directly)
- ACTION (extracted directly)
- INTERNAL is not collected — the human's own business

DM-Left still updates their condition vector after their turn based on their visible outputs and scene events.

Output Format Variants

Standard character (player or NPC):

```
INTERNAL:      [required]
BODY LANGUAGE: [required]
SPOKEN:        [optional — omit if silent]
ACTION:        [optional — omit if still]
```

Creature without language:

```
INTERNAL:      [required — sensation only, no human concepts]
BODY LANGUAGE: [required]
ACTION:        [required — creatures always act]
```

(SPOKEN omitted from format entirely)

Human player (via OpenClaw):

```
[Free text response]
```

Parsed by context builder into BODY LANGUAGE / SPOKEN / ACTION.

Turn Prompt File Naming Convention

Generated prompts follow this naming pattern:

```
turn{N}_{character}_{model}.txt

Examples:
  turn1_aeliana_chatgpt.txt
  turn1_shadowbeast_grok.txt
  turn2_fennel_chatgpt.txt
```

For programmatic generation, prompts are assembled in memory and passed directly to the API — file generation is for testing and debugging only.

Context Builder Responsibilities

```
class ContextBuilder:
    def build_prompt(self,
                     character: Character,
                     scene: Scene,
                     turn: int,
                     prior_actions: list[CharacterAction]
                     ) -> Prompt:

        return Prompt(
            system=self.build_system(character),
            character=self.render_character(character),
            soul=self.render_soul(character.soul),
            condition=self.render_condition(character.condition),
            location=self.render_location(character, scene),
            scene_state=self.filter_scene(character, scene),
            what_changed=self.render_changes(character,
                                             prior_actions,
                                             scene),
            available_actions=self.generate_actions(character,
                                                    scene),
            output_format=self.render_format(character),
        )
```

The context builder is the engine's most complex component. It enforces all information boundaries and produces the filtered view each character is entitled to see.

Validated Design

This prompt structure has been validated through manual play of the Fey Glade scene across multiple LLM providers. Ground truth outputs are archived in:

```
docs/fey_glade_experiment/
```

Key validation findings:

- Information filtering produces dramatically distinct character responses from identical events
- Sequential state propagation (WHAT CHANGED) correctly creates cascading consequences within a turn
- Creature prompts with sensation-only INTERNAL produce phenomenologically rich non-cognitive output
- Available actions list constrains behavior without eliminating agency — characters still surprise
- Condition vectors bias behavior consistently across models without prescribing specific actions

---

Open Questions

1. **Parallel vs sequential LLM calls:** Can characters in the same scene act in parallel if their actions don't interact? (e.g. two characters in different zones with no line of sight to each other)
2. **Reaction actions:** D&D 5e has reaction actions that fire outside normal initiative order. How does this interact with the sequential prompt model?
3. **Perception check adjudication:** When a character takes an OBSERVE action requiring a dice roll, DM-Left adjudicates before the next character's prompt is built. The result may change what the next character observes.
4. **Multi-scene turns:** When multiple scenes are active simultaneously, do all scenes run Turn N before any scene runs Turn N+1, or does each scene run independently?